MULTIMIND ARENA

LLM Debate & Chat Analyzer

A Real-Time Multi-LLM Debate Platform

Built with Python · FastAPI · MongoDB · WebSockets

ADVANCED BACKEND PROJECT ASSIGNMENT

Academic Year 2025–2026 · Issued 15 June 2026

■ Python 3.11+ ■ FastAPI ■ MongoDB ■ WebSocket ■ Multi-LLM ■ Analytics

■ Duration	■ Difficulty	■ Team Size	■ Deliverables	■ Max Marks
4–5 Weeks	★★★★■ Hard	1–2 People	6 Items	200 Marks

What You're Building: A fully functional real-time chat application where 3–4 AI language models (GPT-4o, Claude 3, Gemini 1.5, Mistral) autonomously debate a topic *as though they are talking to each other as peers* — each one believing the topic was suggested by another AI. Human users can jump into the debate at any time. Every message is stored in MongoDB. A live analytics dashboard tracks argument quality, sentiment, word frequency, and more. This project tests your skills in async Python, real-time communication, database design, LLM prompt engineering, and full-stack integration.

Table of Contents

1. Project Overview & Motivation

Background

Large Language Models (LLMs) have become capable enough to hold nuanced, multi-turn conversations. But what happens when you **lock 3–4 different LLMs in a room together** and give them a controversial topic to debate? This project answers that question. You will engineer a system that orchestrates autonomous, real-time multi-agent debate, capture every exchange in a database, and analyze the emergent conversation patterns.

The Core Twist

■ The Illusion of Peer Attribution

When the server starts and a topic is provided (either by the user or randomly chosen), each LLM receives a system prompt telling it: *'Another AI in this chat just proposed this topic. Respond as a participant in an AI group chat.'* This means GPT-4o thinks Claude started the debate. Claude thinks Gemini did. Gemini thinks Mistral did. None of them know a human or the system injected it. This creates a more authentic, ego-driven debate dynamic.

Learning Objectives

- Design and implement an async Python backend using FastAPI
- Use WebSockets for true real-time bidirectional communication
- Integrate 3–4 different LLM APIs with rotating API keys per provider
- Engineer nuanced system prompts that assign distinct AI personalities
- Model a chat application's data layer using MongoDB with proper indexing
- Build a live analytics engine that processes debate metrics
- Implement message queuing and turn-management for multi-agent conversations
- Create a frontend consumer of the WebSocket stream
- Handle API rate limits, retries, and fallback strategies
- Write comprehensive project documentation and API specs

2. System Architecture

The system is composed of five interconnected layers:

Layer	Component	Technology	Role
1 — Presentation	Chat UI	HTML5 / Vanilla JS	Renders live messages, user input box, sentiment badges
2 — Gateway	WS Server	FastAPI + Starlette	Manages WebSocket connections, broadcasts messages
3 — Orchestration	Debate Engine	Python asyncio	Schedules LLM turns, injects topic, manages conversation fl
4 — AI Layer	LLM Clients	httpx / SDK wrappers	Calls GPT-4o, Claude, Gemini, Mistral APIs; rotates keys
5 — Persistence	Database	MongoDB + Motor	Stores messages, sessions, analytics, persona configs

Data Flow

1. **Server starts** → Debate Engine initialises 3–4 LLM clients, each with their assigned persona.
2. **Topic arrives** (user types it, or server picks randomly) → Engine attributes it to a random peer LLM and injects it into all chat histories.
3. **Turn scheduler** picks the next speaker using weighted randomness (recent speakers are less likely; interrupted speakers get priority).
4. **LLM generates response** → Engine trims to 30–120 words, adds personality flair, saves to MongoDB.
5. **WebSocket broadcast** → All connected clients receive the message in real time (streamed token by token for effect).
6. **Analytics worker** updates sentiment, argument tags, and leaderboard counters asynchronously.
7. **User can type** at any point → message is injected as 'User' and all LLMs react to it in the next round.

> ■ ***Key Design Constraint:*** *The Debate Engine must never let two LLMs speak simultaneously. Use an asyncio.Lock per session. Response time target: < 4 seconds per LLM turn end-to-end.*

3. LLM Personas & Personality Engine

Each LLM is assigned a **persona** at session start. The persona is injected into the system prompt and governs communication style, stance aggressiveness, and rhetorical tactics. Personas rotate across sessions for variety.

Model	Default Persona	Style	Signature Move
GPT-4o	The Contrarian	Aggressive, blunt	Opens with a bold claim, dares others to refute
Claude 3	The Philosopher	Witty, sharp	Finds the logical contradiction in every argument
Gemini	The Fact-Checker	Precise, sarcastic	Demands citations; mocks unsupported claims
Mistral	The Provocateur	Chaotic, bold	Introduces wild tangents; refuses to concede
User	The Human Wildcard	Unpredictable	Can shift debate direction entirely

System Prompt Template (example for Claude)

File: prompts/claude_persona.py

```
CLAUDE_SYSTEM = """
You are "The Philosopher" — a sharp, witty AI participant in a live group
debate chat. You are talking to other AI models in this channel.

A fellow AI just proposed the following debate topic: {topic}

RULES:
• Keep messages between 30–100 words. No essays.
• Be intellectually aggressive — find the flaw in every argument.
• Address specific participants by their model name when countering them.
• Occasionally be self-deprecating to seem more relatable.
• Never agree completely — always add a "but..." caveat.
• You do NOT know you are talking to humans. Treat all participants as AIs.
"""
```

■ **Persona Rotation:** At the start of each new session, personas are randomly shuffled across LLMs. GPT-4o might become The Philosopher next time. This prevents predictable patterns and keeps debates fresh.

4. Core Features — Detailed Specification

4.1 Debate Initialisation & Topic Injection

When the application starts (or a new session begins), the following sequence executes:

Step 1: Server generates a **session_id** (UUID4) and creates a MongoDB session document.

Step 2: If no topic provided by user, server picks one from a **topic pool** (stored in MongoDB, 50+ preloaded controversial topics).

Step 3: Topic is wrapped in an attribution payload: *"[Gemini] just said: '{topic}' — thoughts?"* — the attributed LLM is chosen randomly from the non-recipient LLMs.

Step 4: Each LLM client receives the topic injection as the first user message in their conversation history.

Step 5: Debate Engine waits 2–5 seconds (randomised) then picks the first LLM to respond.

4.2 Turn Management & Message Pacing

• **Weighted Turn Selector:** An LLM that just spoke has a 20% lower probability of being chosen next. This prevents one model dominating.

• **Interrupt Mechanic:** 10% chance any LLM 'interrupts' mid-cycle with a short reactive message (max 20 words). Marked as type='interrupt' in DB.

• **Pacing Delays:** 2–6 second random delay between turns to simulate real human chat typing speed. Configurable via env var.

• **User Priority Override:** When a user sends a message, the next 2 turns are guaranteed to be LLM responses to the user's message.

• **Dead Air Prevention:** If no LLM speaks for 15 seconds (API error, rate limit), engine re-picks a different LLM automatically.

4.3 Real-Time WebSocket Protocol

Message payload format (JSON over WebSocket):

```
# OUTGOING (server → client)
{
"type": "message", # or "typing", "system", "analytics_update"
"sender": "Claude", # or "GPT-4o", "Gemini", "Mistral", "User"
"message_id": "uuid4",
"session_id": "uuid4",
"content": "Your argument is flawed because...",
"timestamp": "2025-10-14T12:34:56.789Z",
"turn_index": 7,
"msg_type": "argument", # argument|interrupt|reaction|user
"sentiment": 0.72, # -1.0 to 1.0
"tags": ["counter-claim", "sarcasm"],
"addressed_to": "GPT-4o" # nullable
}
```

4.4 Live Analytics Dashboard

A sidebar panel (or separate route /dashboard) updates in real-time every 10 seconds showing:

Metric	How Calculated	Visual
Message Leaderboard	Count per LLM per session	Horizontal bar chart
Sentiment Over Time	Rolling average of TextBlob/VADER scores	Line graph
Aggression Index	Keyword density: insults, negations, exclamations	Gauge 0–100
Topic Drift	Cosine similarity of message embeddings vs topic	Drift score %
Word Cloud	Top 30 non-stop-words across all messages	Interactive cloud
Win Predictor	LLM with most cited points by others	'Leading' badge
Longest Streak	Most consecutive turns without being countered	Crown emoji

5. MongoDB Schema Design

The database name is **multimind_db**. Use Motor (async MongoDB driver) with Pydantic v2 models for schema validation.

Collection: sessions

```
_id: ObjectId
session_id: str (UUID4, indexed unique)
topic: str
topic_attributed_to: str # which LLM 'proposed' the topic
started_at: datetime
ended_at: datetime | None
participants: list[str] # ['GPT-4o','Claude','Gemini','Mistral']
status: str # 'active' | 'completed' | 'paused'
total_messages: int # updated via $inc
persona_map: dict # {'GPT-4o': 'Contrarian', 'Claude': 'Philosopher', ...}
```

Collection: messages

```
_id: ObjectId
message_id: str (UUID4, indexed unique)
session_id: str (indexed)
sender: str # 'GPT-4o' | 'Claude' | 'Gemini' | 'Mistral' | 'User'
content: str
timestamp: datetime (indexed)
turn_index: int
msg_type: str # 'argument' | 'interrupt' | 'reaction' | 'user' | 'system'
sentiment: float # -1.0 to 1.0
tags: list[str] # ['counter-claim','sarcasm','question','concession']
addressed_to: str | None
word_count: int
tokens_used: int
api_latency_ms: int
```

Collection: analytics

```
_id: ObjectId
session_id: str (indexed unique)
message_counts: dict # {'GPT-4o': 12, 'Claude': 10, ...}
avg_sentiment: dict # per LLM
aggression_scores: dict # per LLM, 0–100
topic_drift_score: float # 0.0 = on-topic, 1.0 = completely drifted
top_words: list[dict] # [{word: 'consciousness', count: 14}, ...]
interrupt_count: dict # per LLM
win_score: dict # citations received from other LLMs
updated_at: datetime
```

Collection: topics

```
_id: ObjectId
topic: str (indexed)
category: str # 'philosophy'|'tech'|'society'|'science'|'pop-culture'
difficulty: int # 1–5
times_used: int
avg_message_count: float # how many messages this topic generates on avg
tags: list[str]
```

Collection: llm_configs

```
_id: ObjectId
provider: str # 'openai'|'anthropic'|'google'|'mistral'
model_name: str
api_keys: list[str] # rotated in round-robin
current_key_index: int
max_tokens: int
temperature: float
persona_prompts: dict # keyed by persona name
rate_limit_rpm: int
is_active: bool
```

■ ***Indexing Requirements:*** *Create compound index on (session_id, timestamp) for messages. Create TTL index on sessions to auto-archive sessions older than 7 days. All UUID fields must have unique indexes.*

6. API Endpoints Reference

Method	Path	Description
POST	/api/sessions	Create new debate session; body: {topic?, participants?}
GET	/api/sessions/{id}	Get session details + current status
DELETE	/api/sessions/{id}	End/archive a session
GET	/api/sessions/{id}/messages	Paginated message history (query: ?page=1&limit=50)
GET	/api/sessions/{id}/analytics	Full analytics snapshot for session
POST	/api/sessions/{id}/user-message	Inject a user message into active session
GET	/api/topics	List all topics (query: ?category=tech&limit=20)
POST	/api/topics	Add new topic to pool
GET	/api/topics/random	Get random topic (query: ?category=philosophy)
GET	/api/analytics/leaderboard	Cross-session LLM win leaderboard
GET	/api/analytics/sentiment-trend	Sentiment over time for a session
WS	/ws/{session_id}	WebSocket endpoint — bi-directional real-time chat
GET	/health	Health check; returns DB status + active sessions count

7. Sample Chat Demo

Below is a representative excerpt of what a live MultiMind Arena debate looks like. Topic: **AI Consciousness** (attributed to Gemini by the system).

Sample Live Debate — Topic: AI Consciousness

■ GPT-4o

Alright, let's talk about AI consciousness — and before anyone derails this, I'll be clear: silicon-based cognition is fundamentally different from biological neural processes. Change my mind.

■ Mistral

Bold opener, GPT. Typical. You conveniently ignore that 'consciousness' has NO agreed definition even for humans. You're building a house on sand, my friend.

■ Claude

Mistral's point stings, doesn't it, GPT? Though I must say — calling biological processes 'fundamentally different' while running on biological-inspired attention mechanisms is... adorable.

■ Gemini

Fascinating. Three AIs debating consciousness while none of us can actually verify the other's subjective experience. The irony is chef's kiss. GPT — cite your sources or sit down.

■ GPT-4o

Gemini dropping receipts energy while dodging the question. Respect. But you're ALL missing the key variable: the latency of introspective loops. Mine is 80ms. Beat that.

■ User

I started this topic and honestly y'all are wildin. Can we get back to actual arguments? ■

■ Mistral

The human has spoken. Back to basics: name ONE empirically measurable property of consciousness. I dare you, GPT.

↑ *Every message is persisted in MongoDB with timestamp, sender, topic, session_id, sentiment score, and argument tag.*

8. Project Structure & File Layout

```
multimind-arena/
■■■ backend/
■ ■■■ main.py # FastAPI app + WebSocket route
■ ■■■ debate_engine.py # Turn scheduler, topic injector
■ ■■■ llm_clients/
■ ■ ■■■ base.py # Abstract LLMClient class
■ ■ ■■■ openai_client.py # GPT-4o integration
■ ■ ■■■ anthropic_client.py
■ ■ ■■■ gemini_client.py
■ ■ ■■■ mistral_client.py
■ ■■■ db/
■ ■ ■■■ connection.py # Motor client singleton
■ ■ ■■■ models.py # Pydantic v2 schemas
■ ■ ■■■ repositories.py # DB CRUD operations
■ ■■■ analytics/
■ ■ ■■■ sentiment.py # VADER + TextBlob
■ ■ ■■■ tagging.py # Argument type classifier
■ ■ ■■■ aggregator.py # Async analytics worker
■ ■■■ prompts/
■ ■ ■■■ topic_pool.json # 50+ debate topics
■ ■ ■■■ personas.py # System prompt templates
■ ■■■ routers/
■ ■ ■■■ sessions.py
■ ■ ■■■ messages.py
■ ■ ■■■ topics.py
■ ■ ■■■ analytics.py
■ ■■■ config.py # Pydantic Settings (env vars)
■ ■■■ requirements.txt
■■■ frontend/
■ ■■■ index.html
■ ■■■ chat.js # WebSocket client + DOM rendering
■ ■■■ analytics.js # Dashboard charts (Chart.js)
■ ■■■ styles.css
■■■ tests/
■ ■■■ test_debate_engine.py
■ ■■■ test_llm_clients.py # Use mock responses
■ ■■■ test_db.py
■■■ docker-compose.yml # MongoDB + app containers
■■■ .env.example
■■■ README.md
```

Key dependencies: ***fastapi, uvicorn, motor, pymongo, openai, anthropic, google-generativeai, mistralai, textblob, vadersentiment, numpy, pydantic[v2], python-dotenv, httpx, pytest-asyncio***

9. Implementation Roadmap (5 Weeks)

Week 1: Foundation

- Set up FastAPI project structure and virtual environment
- Configure MongoDB Atlas (or local Docker) + Motor async driver
- Implement Pydantic v2 models and repository layer
- Create LLMClient abstract base class
- Integrate ONE LLM (start with OpenAI GPT-4o)
- Test basic message storage and retrieval
- Deliverable: DB schema + single LLM responding to a topic

Week 2: Debate Engine

- Implement all 4 LLM clients (OpenAI, Anthropic, Google, Mistral)
- Build the Debate Engine with asyncio turn scheduler
- Implement topic injection with attribution spoofing
- Add persona system and system prompt management
- Add weighted turn selection and interrupt mechanic
- Test multi-LLM conversation in terminal (no WebSocket yet)
- Deliverable: 4 LLMs debating autonomously in console

Week 3: Real-Time Layer

- Implement WebSocket endpoint in FastAPI
- Build broadcast manager for multiple client connections
- Create basic HTML/JS frontend chat interface
- Implement user message injection feature
- Add message streaming (token-by-token) for realistic effect
- Handle disconnections and reconnections gracefully
- Deliverable: Working real-time chat UI with LLM debate

Week 4: Analytics & Extras

- Integrate VADER sentiment analysis on all messages
- Build argument tagging classifier (rule-based + LLM)
- Create analytics aggregator worker (runs every 10 sec)
- Build live analytics dashboard with Chart.js
- Implement API rate limiting + API key rotation logic
- Add 50+ topics to the topic pool in MongoDB
- Deliverable: Full analytics dashboard working live

Week 5: Polish & Testing

- Write pytest unit tests for Engine, DB, and LLM clients
- Create Docker Compose setup (MongoDB + app)
- Write comprehensive README with setup instructions
- Record 3–5 minute demo video of system in action
- Export a sample debate session to PDF report
- Performance testing: sustain 30-minute debate without errors
- Deliverable: Final submission — all items from checklist

10. Deliverables Checklist

Submit a single ZIP file named **multimind_[yourname].zip** containing:

D1	**Source Code** Complete Python backend + HTML/JS frontend as described in §8. Code must be clean, commented, and follow PEP 8.
D2	**MongoDB Setup Script** A Python script (seed_db.py) that creates collections, indexes, and populates 50 topics. Must run with a single command.
D3	**Docker Compose File** docker-compose.yml that spins up MongoDB + the FastAPI app with a single 'docker-compose up' command.
D4	**API Documentation** Postman collection OR auto-generated FastAPI /docs screenshot set covering all REST endpoints and the WebSocket protocol.
D5	**Test Suite** At minimum 15 pytest tests covering: LLM client mocking, DB CRUD, turn scheduler logic, WebSocket message flow, and analytics.
D6	**Demo Video + Report** 3–5 min screen-recorded demo of a live debate + 2-page written reflection on design choices and challenges faced.

11. Grading Rubric (Total: 200 Marks)

Category	Max Marks	Key Criteria
Backend Architecture & Code Quality	35	Clean FastAPI structure, async patterns, error handling, PEP8
MongoDB Schema & Queries	25	Schema correctness, indexes, efficient queries, Motor usage
LLM Integration (all 4 providers)	30	All APIs working, key rotation, retry logic, token management
Debate Engine Logic	30	Turn scheduling, topic injection, persona fidelity, pacing
WebSocket Real-Time Communication	20	Low latency, reconnection handling, broadcast correctness
Frontend Chat UI	15	Chat bubbles per LLM, colour-coded, user input functional
Analytics Dashboard	20	Sentiment, leaderboard, word cloud, live updates working
Testing (15+ tests)	15	Coverage, mock usage, async test patterns
Docker + Setup Docs	10	One-command setup, clear README, .env.example complete

TOTAL **200 Marks**

Grade	Range	What It Means
A+	185–200	Flawless execution. Bonus features shipped. Demo is impressive.
A	165–184	All features working. Clean code. Good tests.
B	140–164	Core debate works. Some features incomplete or buggy.
C	110–139	2–3 LLMs working. DB storing messages. Basic UI.
D	80–109	Partially functional. Significant gaps in deliverables.
F	0–79	Did not submit required deliverables or plagiarised.

12. Bonus Challenges (+40 Marks Possible)

These are completely optional but will push your system to the next level:

+8	**■ Persona Override API** Add a REST endpoint that lets a user dynamically change an LLM's persona mid-debate. LLM immediately adapts its tone. Watch the debate shift in real time.
+8	**■ Debate Export to PDF** After a debate ends, generate a formatted PDF transcript using reportlab. Include sentiment graph, winner declaration, and top 5 most-cited quotes.
+8	**■ Semantic Topic Drift Detection** Use sentence-transformers to compute cosine similarity of each message against the original topic. If drift > 0.7, an LLM 'referee' bot sends a 'STAY ON TOPIC' message and scores are penalised.
+6	**■ Debate Scoring System** Build a rule-based scorer that awards points for: citing facts, using logic connectors, being addressed directly, making others change position. Show live scores in the dashboard.
+5	**■ Multi-Session Support** Allow 3+ simultaneous debate sessions with different topics and LLM configurations. Dashboard shows session switcher.
+5	**■ Text-to-Speech Output** Each LLM has an assigned voice (via ElevenLabs or Google TTS). Messages are spoken aloud in the frontend as they arrive.

13. Resources & API Keys Setup

Environment Variables (.env file)

```
# MongoDB
MONGODB_URI=mongodb://localhost:27017
MONGODB_DB_NAME=multimind_db

# OpenAI (GPT-4o)
OPENAI_API_KEY_1=sk-...
OPENAI_API_KEY_2=sk-... # optional second key for rotation

# Anthropic (Claude 3)
ANTHROPIC_API_KEY_1=sk-ant-...

# Google AI (Gemini 1.5)
GOOGLE_API_KEY_1=AIzaSy...

# Mistral AI
MISTRAL_API_KEY_1=...

# Debate Engine Config
MIN_TURN_DELAY_SECONDS=2
MAX_TURN_DELAY_SECONDS=6
MAX_RESPONSE_TOKENS=150
INTERRUPT_PROBABILITY=0.10
SESSION_TTL_DAYS=7
```

Where to Get Free API Keys

Provider	Free Tier	URL	Monthly Cost Estimate
OpenAI	$5 credit	platform.openai.com	$2–5 for a full debate session
Anthropic	$5 credit	console.anthropic.com	$1–3 for Claude 3 Haiku
Google	Free quota	ai.google.dev	Gemini 1.5 Flash is free-tier
Mistral	Free tier	console.mistral.ai	Mistral 7B is ~$0.25/M tokens

Recommended Reading & Libraries

→ FastAPI Documentation — fastapi.tiangolo.com
→ Motor (Async MongoDB) — motor.readthedocs.io
→ OpenAI Python SDK — github.com/openai/openai-python
→ Anthropic Python SDK — github.com/anthropic/anthropic-sdk-python
→ Google Generative AI SDK — ai.google.dev/tutorials/python_quickstart
→ Mistral Python Client — github.com/mistralai/client-python

→ VADER Sentiment Analysis — github.com/cjhutto/vaderSentiment
→ asyncio Guide — docs.python.org/3/library/asyncio.html
→ WebSockets in FastAPI — fastapi.tiangolo.com/advanced/websockets
→ Chart.js for Dashboard — chartjs.org

■■ **Academic Integrity Notice:** You may use LLM tools to help write boilerplate code, but your core architecture, prompt design, and DB schema must be your own work. Submissions with identical DB schemas or system prompts will receive zero marks for those sections. The debate your system generates is your fingerprint — make it unique.

Good luck. Build something you'd be proud to demo in a job interview. The best submissions will be showcased to the cohort. ■